TOPIC 3

Functions, Modules and Program Organisation

Object-Oriented Programming Using Python

From one long script → reusable functions → organised modules → maintainable programs

Learning Outcomes

By the end of Topic 3, students should be able to:

Decompose a programming problem into small, reusable functions.

Define and call functions using appropriate parameters, arguments and return values.

Explain variable scope and manage data flow between functions.

Use built-in, standard-library and user-defined modules to organise programs.

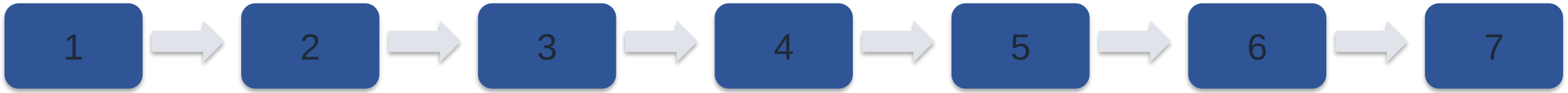
Use selected Python libraries to solve engineering-oriented problems.

Organise a larger Python program across multiple files and explain why modularity matters.

Topic Roadmap

Overview

The journey from basic functions to multi-file Python programs



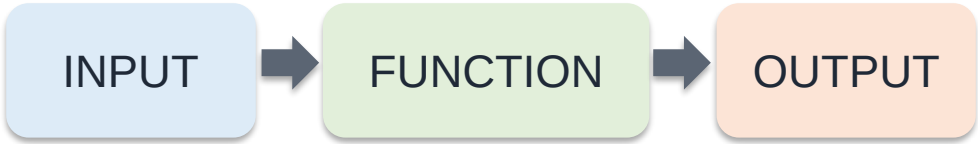
Why functions? **Defining & calling** **Parameters & returns** **Scope** **Modules & imports** **Packages & pip** **Program organisation**

Core idea: divide a complex problem into understandable, testable and reusable parts.

Why Do We Need Functions?

Functions reduce repetition and make programs easier to understand.

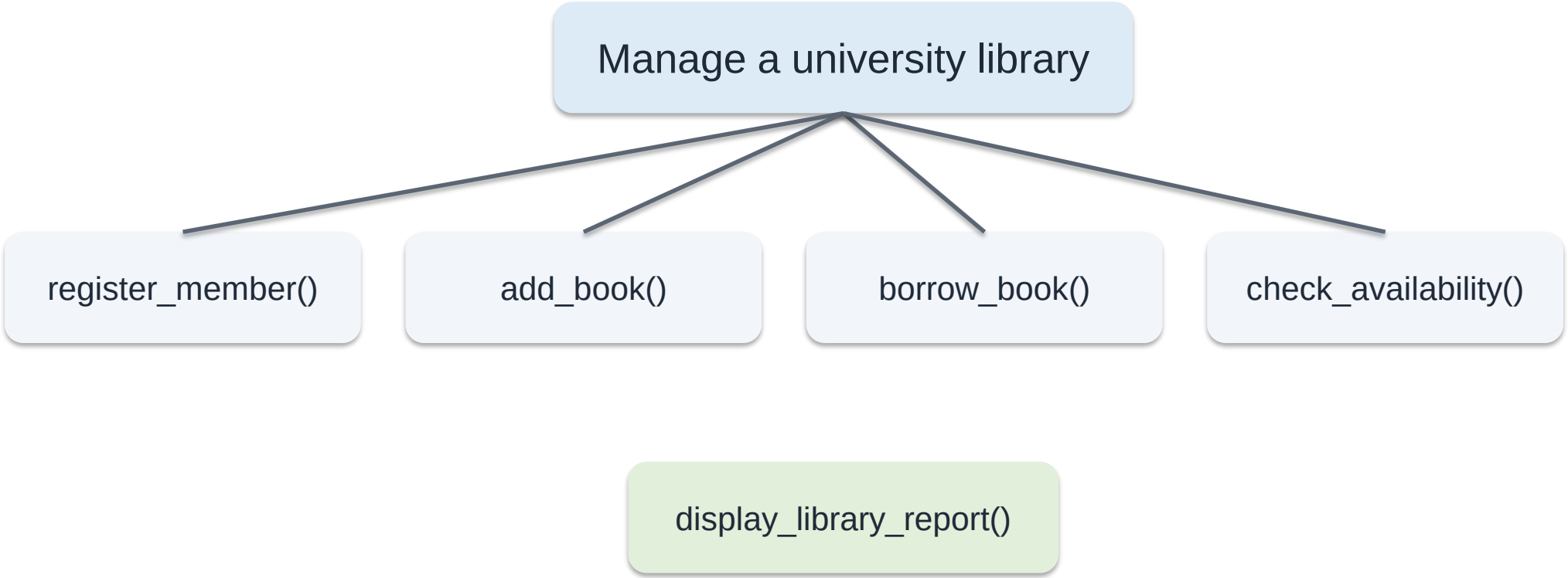
Without functions, a large program can become one long sequence of statements.
Repeated code is difficult to correct: the same change may be required in many places.
A function gives a meaningful name to a task, e.g. borrow_book(), convert_temperature(), validate_voltage().
Functions support decomposition: solve one large problem as several smaller problems.
They improve reuse, testing, debugging, teamwork and later object-oriented design.



Think of a function as a small machine: values go in → work happens → a result may come out.

Problem Decomposition

Break a large engineering task into smaller responsibilities.



Each function has one clear task.

The main program coordinates the functions.

Defining a Function

Use def, a function name, parentheses and an indented body.

Python

```
def greet():
    print("Welcome to Python!")

# Calling the function
greet()
```

`def` tells Python that a function is being defined.

`greet` is the function name.

`()` holds parameters when the function needs input.

`:` begins the function body.

The indented statements belong to the function.

Defining a function does NOT execute it; the function runs when it is called.

Naming Functions Well

A function name should communicate what the function does.

Use meaningful verbs or verb phrases: calculate_area(), read_sensor(), save_record().

Use lowercase words separated by underscores (snake_case).

Do not use spaces, hyphens or Python keywords.

Avoid vague names such as do_it(), thing() or function1().

Keep a function focused: one clear responsibility is easier to name.

Good
calculate_resistance()

Poor
calc()

A good name often removes the need for a comment explaining the function.

Parameters: Giving Data to a Function

Parameters are variables declared in the function definition.

Python

```
def calculate_fine(days_late, rate):
    fine = days_late * rate
    print(f"Fine = UGX {fine}")

calculate_fine(3, 1000)
calculate_fine(5, 500)
```

`days_late` and `rate` are parameters.
`3` and `1000` are arguments supplied during the first call.
The same function can process different values.
Parameters make functions reusable instead of hard-coding data.

Parameter = placeholder in the definition
Argument = actual value supplied in the call

Positional Arguments

With positional arguments, order determines which value goes to which parameter.

Python

```
def calculate_bmi(mass, height):
    return mass / (height ** 2)

bmi = calculate_bmi(70, 1.75)
print(bmi)
```

`70` is assigned to `mass` because it appears first.

`1.75` is assigned to `height` because it appears second.

Changing the order can change the meaning or produce incorrect results.

Use positional arguments when the meaning and order are obvious.

Keyword Arguments

Keyword arguments identify parameters by name.

Python

```
def student_record(name, year, programme):
    print(name, year, programme)

student_record(
    programme="Engineering",
    name="Amina",
    year=2
)
```

Keyword arguments use parameter_name=value.

The order can be changed because each value is explicitly labelled.

They can make function calls easier to read.

Python allows positional and keyword arguments together, but positional arguments normally come first.

Default Parameters

A default value is used when the caller does not provide that argument.

Python

```
def calculate_fine(days_late, rate=1000):
    return days_late * rate

print(calculate_fine(3))
print(calculate_fine(3, 1500))
```

`rate=1000` gives `rate` a default value.
First call: rate is omitted → Python uses 1000.
Second call: 1500 is supplied → it replaces the default.
Required parameters should normally appear before parameters with defaults.

Returning a Result

return sends a value back to the code that called the function.

Python

```
def calculate_fine(days_late, rate):
    return days_late * rate

fine = calculate_fine(4, 1000)
print(f"Fine = UGX {fine}")
```

`return` ends the function and sends a result back.

The caller can store, print, compare or pass the returned value elsewhere.

A function that only prints a result is less reusable than one that returns it.

Good design often separates calculation from presentation.

calculate_fine() calculates.
print() displays.

print() Is Not the Same as return

This distinction is essential for beginners.

Python

```
# Version A: prints only
def area_a(length, width):
    print(length * width)

# Version B: returns a value
def area_b(length, width):
    return length * width

result = area_b(5, 3)
cost = result * 20000
```

`print()` sends text to the screen.

`return` sends a value back to the calling code.

A returned value can participate in later calculations.

Ask students: Which version is more reusable? Why?

Rule of thumb: return data; let another part of the program decide how to display it.

Returning Multiple Values

Python can return several values, commonly unpacked into variables.

Python

```
def loan_summary(days_late, rate):
    fine = days_late * rate
    status = "Overdue" if days_late > 0 else "On time"
    return fine, status

fine, status = loan_summary(3, 1000)
print(fine)
print(status)
```

The function returns two values.

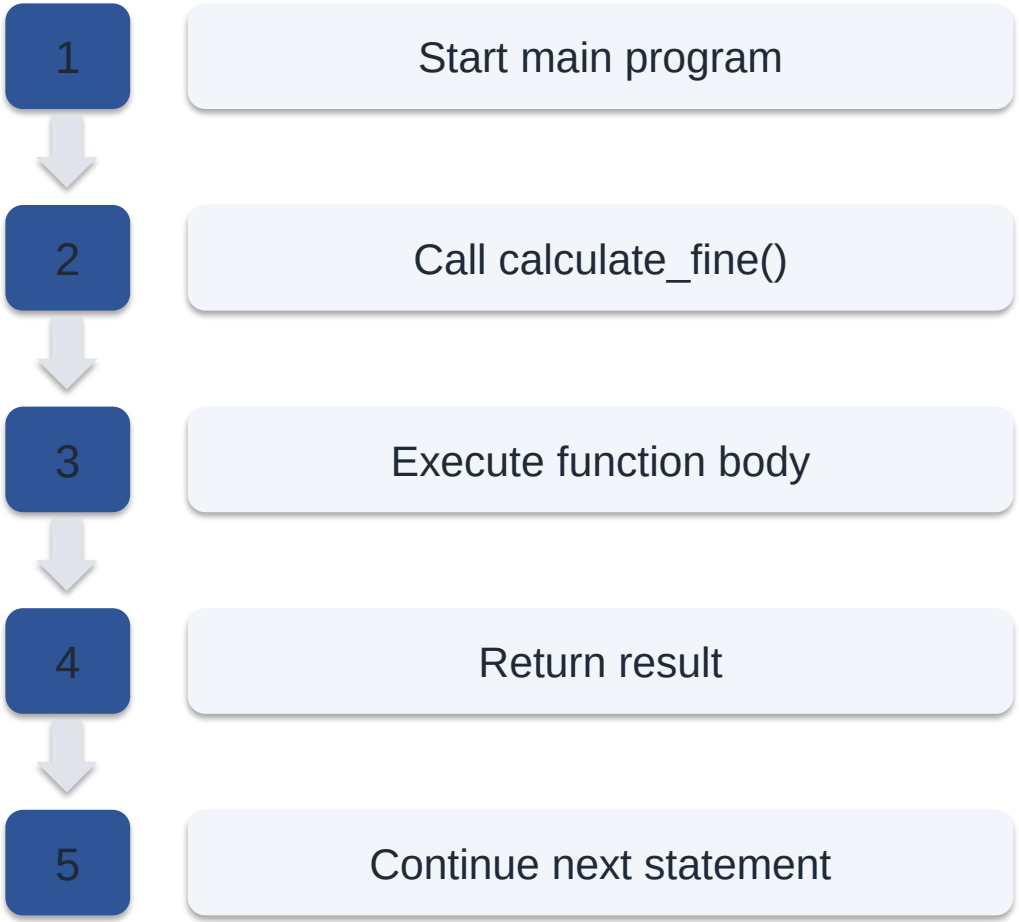
`p, r = ...` unpacks them into two variables.

Internally, Python commonly represents the returned group as a tuple.

Use multiple returns when the results naturally belong to one computation.

How Function Calls Change Program Flow

Execution jumps into the function, then returns to the caller.



Python

```
def calculate_fine(days_late, rate):
    return days_late * rate

days = 3
rate = 1000
fine = calculate_fine(days, rate)
print(fine)
```

Variable Scope

Scope determines where a variable can be accessed.

GLOBAL SCOPE
Defined outside functions

LOCAL SCOPE
Defined inside a function

A local variable normally exists only while its function is executing.

A global variable can be accessed from a wider part of the module.

Prefer passing data through parameters and return values rather than relying heavily on globals.

Local and Global Variables: Example

Trace which variable is visible at each point.

Python

```
library_name = "UCU Library"   # global

def show_loan():
    book_title = "Python Basics"   # local
    print(library_name)
    print(book_title)

show_loan()
print(library_name)
# print(book_title)   # Error!
```

`library_name` is defined outside the function and can be read inside it.
`book_title` is local to show_loan().
Trying to access `book_title` outside the function causes a NameError.
Local variables help prevent accidental interference between functions.

Why Avoid Unnecessary Global Variables?

Globals can make data flow difficult to understand.

Python

```
# Harder to reason about
available_copies = 5

def borrow_book():
    global available_copies
    available_copies -= 1
```

Python

```
# Clearer data flow
def borrow_book(available_copies):
    return available_copies - 1

available_copies = borrow_book(5)
```

Prefer explicit INPUT → PROCESS → OUTPUT when practical.

Variable-Length Arguments: *args and **kwargs

Useful when the number of arguments is not fixed.

Python

```
def average(*values):
    return sum(values) / len(values)

print(average(10, 20))
print(average(10, 20, 30, 40))
```

Python

```
def show_settings(**settings):
    for key, value in settings.items():
        print(key, value)

show_settings(unit="V", precision=2)
```

`*args` collects extra positional arguments, usually as a tuple.

`**kwargs` collects extra keyword arguments, usually as a dictionary.

Lambda Functions

A lambda is a small anonymous function written as one expression.

Python

```
# Normal function
def square(x):
    return x * x

# Lambda equivalent
square2 = lambda x: x * x

print(square2(5))
```

Syntax: lambda parameters: expression

Useful for short operations passed to functions such as sorted().

Not suitable for complex multi-step logic.

For teaching, emphasise readability over cleverness.

Built-in Functions

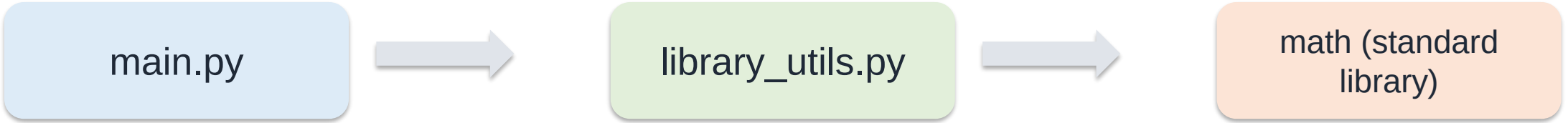
Python already provides many useful functions.

len()	sum()	min()/max()	round()
number of items	total	extremes	round a number
abs()	sorted()	type()	range()
absolute value	sorted copy	object type	integer sequence

Before writing a new function, ask: does Python already provide one?

What Is a Module?

A module is a Python file containing reusable code.



Modules help separate responsibilities and reduce the size of individual files.

A module can contain functions, variables and later classes.

Modules support reuse: write once, import where needed.

Python itself is distributed with a rich standard library of modules.

Importing Modules

Python provides several import styles.

Python

```
import math
print(math.sqrt(81))

from math import sqrt
print(sqrt(81))

import statistics as stats
print(stats.mean([10, 20, 30]))
```

`import math` keeps the module name visible: math.sqrt().

`from math import sqrt` imports a selected name directly.

`as` creates an alias, e.g. statistics as stats.

Avoid `from module import *` in normal programs because names become unclear.

Using Standard Modules in a Library System

The standard math module provides mathematical functions and constants.

Python

```
from datetime import date, timedelta

borrow_date = date.today()
loan_period = 14
return_date = borrow_date + timedelta(days=loan_period)

print(f"Borrowed: {borrow_date}")
print(f"Due date: {return_date}")
```

`math.pi` — π

`math.sqrt(x)` — square root

`math.sin(), cos(), tan()` — trigonometry

`math.radians()` / `degrees()` — angle conversion

`math.log()` / `log10()` — logarithms

`math.ceil()` / `floor()` — rounding direction

The random Module

Useful for simulation, sampling and generating test data.

Python

```
import random

books = ["Python Basics", "Networks", "Databases", "AI"]
book = random.choice(books)
print(f"Selected book: {book}")
```

`random.randint(a, b)` generates an integer in the inclusive range.

`random.uniform(a, b)` generates a floating-point value.

`random.choice(sequence)` selects one item.

Pseudo-random values are useful for demonstrations, but are not automatically suitable for security-sensitive tasks.

The statistics Module

Summarise a set of measurements using descriptive statistics.

Python

```
import statistics

loan_days = [7, 14, 10, 21, 12]

print(statistics.mean(loan_days))
print(statistics.median(loan_days))
print(statistics.stdev(loan_days))
```

`mean()` — arithmetic average
`median()` — middle value
`mode()` — most common value
`stdev()` — sample standard deviation
Excellent for introductory analysis of library loan durations.

The datetime Module

Programs often need to represent dates and times.

Python

```
from datetime import datetime

now = datetime.now()
print(now)
print(now.year)
print(now.strftime("%d-%m-%Y %H:%M"))
```

Record when a book was borrowed or returned.
Timestamp transactions or system events.
Calculate durations and deadlines.
Format dates for users.
`datetime` is useful later in database-backed applications.

Creating Your Own Module

Move reusable functions into a separate .py file.

library_utils.py

```
# library_utils.py

def power(voltage, current):
    return voltage * current

def resistance(voltage, current):
    return voltage / current
```

main.py

```
# main.py
import library_utils

fine = library_utils.calculate_fine(3)
available = library_utils.is_available(2)

print(fine)
print(available)
```

Both files should be in the same project folder for this simple example.

The __name__ == '__main__' Pattern

Control what runs when a file is executed versus imported.

Python

```
def main():
    member = "Amina"
    book = "Python Basics"
    print(f"{member} borrowed {book}")

if __name__ == "__main__":
    main()

#
```

When a file is run directly, `__name__` is set to `'__main__'`.

When the file is imported, its module name is used instead.

This pattern prevents demonstration or startup code from running automatically during import.

It gives larger Python programs a clear entry point.

From Modules to Packages

Organisation

A package groups related modules into a directory structure.

library_system/

main.py

calculations.py

sensors.py

reports.py

A module is usually one `.py` file.

A package organises multiple related modules.

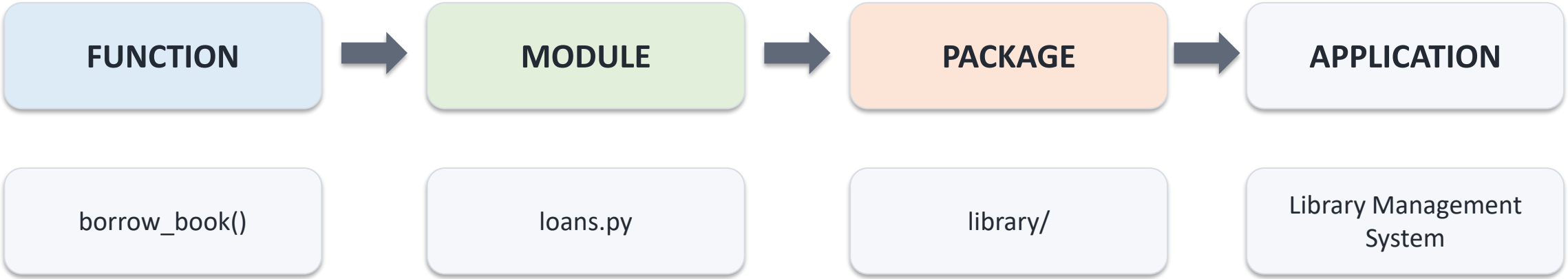
Packages become increasingly useful as projects grow.

Later OOP projects can separate models, services, repositories and user-interface code.

Organisation is not decoration—it controls complexity.

From Functions to Modules to Packages

How we organise a growing Python Library Management System



As programs grow, related code is grouped instead of keeping everything in one file.

Step 1 — Start with Functions

A function is a named, reusable unit of behaviour.

main.py

```
def calculate_fine(days_late, rate=1000):
    return days_late * rate

def is_available(copies):
    return copies > 0
```

- A small program can initially keep functions in main.py.
- As functions increase, main.py becomes long and difficult to ma
- Move related functions into separate Python files.
- Those Python files are called modules.

Function = one reusable task

Step 2 — Create a Module

A module is one Python .py file containing related code.

loans.py

```
# loans.py

def calculate_fine(days_late, rate=1000):
    return days_late * rate

def borrow_book(member, book):
    return f"{member} borrowed {book}"
```

main.py

```
# main.py
import loans

fine = loans.calculate_fine(4)
print(fine)
```

MODULE = one .py file

Why One Module Is Eventually Not Enough

Our Library Management System keeps growing.

books.py	members.py	loans.py	reports.py
add_book() search_book() check_availability()	register_member() search_member()	borrow_book() return_book() calculate_fine()	display_report() loan_summary()

These modules are related. We can group them inside one package.

Step 3 — Create a Package

A package is a folder that groups related Python modules.

Project structure

```
library_project/
│
├── main.py
│
└── library/
    ├── __init__.py
    ├── books.py
    ├── members.py
    ├── loans.py
    └── reports.py
```

- library is the package.
- books.py, members.py, loans.py and reports.py are modules.
- main.py is the program entry point in this simple design.
- __init__.py marks the folder as a regular Python package and can initially be empty.

Package = folder of related modules

Creating the Package in VS Code

No special wizard is required—create the folders and files yourself.

1	**Create project folder**	library_project
2	**Create package folder**	library
3	**Create special file**	__init__.py
4	**Create modules**	books.py • members.py • loans.py • reports.py
5	**Create main program**	main.py

VS Code: Explorer → New Folder / New File

What Is __init__.py?

The filename begins and ends with TWO underscores: __init__.py

Package

```
library/
    __init__.py
    books.py
    members.py
    loans.py
    reports.py
```

- For beginners, include __init__.py in every package you create.
- It may be completely empty.
- It makes the package structure explicit and can contain package initialisation code.
- Modern Python also supports namespace packages without it, but that is not necessary here.

Beginner rule: Package folder → include __init__.py

Importing from a Package

Use the package name, module name and function name.

library/loans.py

```
# library/loans.py

def calculate_fine(days_late, rate=1000):
    return days_late * rate
```

main.py

```
# main.py
from library.loans import calculate_fine

fine = calculate_fine(5)
print(f"Fine = UGX {fine}")
```

from library . loans import calculate_fine

package module function

Different Ways to Import

Choose a clear import style.

main.py

```
# Import module
from library import loans
fine = loans.calculate_fine(5)

# Import one function
from library.loans import calculate_fine
fine = calculate_fine(5)

# Import with alias
from library import loans as ln
fine = ln.calculate_fine(5)
```

- Module-style imports make the source obvious.
- Direct function imports are concise.
- Aliases can shorten long names.
- Avoid import * because names become harder to trace.

How the Pieces Fit Together

A package creates a clear hierarchy.

Library Management System

library package

books.py	**members.py**	**loans.py**	**reports.py**
add_book()	register_member()	borrow_book()	display_report()

Application → Package → Modules → Functions

Module vs Package — Do Not Confuse Them

This distinction is a common exam question.

MODULE

- One Python file
- Usually ends in .py
- Example: loans.py
- Contains functions, variables and later classes

PACKAGE

- A directory/folder
- Groups related modules
- Example: library/
- In this course, include __init__.py

Analogy: Module = one book | Package = bookshelf containing related books

Class Exercise

Build the package structure before writing the full application.

Create this

```
library_project/
├── main.py
└── library/
    ├── __init__.py
    ├── books.py
    ├── members.py
    ├── loans.py
    └── reports.py
```

- In loans.py, create calculate_fine(days_late, rate=1000).
- In books.py, create is_available(copies).
- Import both functions into main.py.
- Call the functions and display their results.
- Identify the package, modules and functions.

pip and Third-Party Packages

Organisation

pip installs Python packages that are not part of the standard library.

Terminal

```
# In the VS Code terminal
python -m pip install package_name

# See installed packages
python -m pip list
```

Standard library modules come with Python; third-party packages are installed separately.

`python -m pip ...` explicitly runs pip using the selected Python interpreter.

Only install packages from trusted sources and only when needed.

Examples students may meet later: Django, pytest, database drivers and data-science libraries.

Virtual Environments

A virtual environment is an isolated Python environment created for a particular project.
It allows that project to have its own Python packages and package versions without interfering with other Python projects on the same computer.

Suppose we have two projects:

```
Computer
│
├── Library Management System
│       └── requires Django 5.x
│
└── Old Student System
        └── requires an older Django version
```

Different projects may require different package versions.

A virtual environment keeps project dependencies separate.

VS Code can select the Python interpreter from `.venv`.

```
Computer
│
├── Library Project
│       └── .venv/
│               └── its packages
│
└── Student Project
        └── .venv/
                └── its packages
```

Virtual Environments

Giving each Python project its own isolated working environment

A virtual environment isolates the Python packages used by one project from other projects.

Why Do We Need a Virtual Environment?

Different projects may need different packages or package versions.

Library Project

Needs its own dependencies
Example: a current Django version

Old Student System

May depend on different versions
or completely different packages

Without isolation

Possible dependency conflicts

With .venv

Each project manages its own dependencies.

What Does a Virtual Environment Contain?

It creates a project-specific Python environment.

Project structure

```
library_project/
│
├── .venv/          ← virtual environment
├── main.py
└── library/
    ├── __init__.py
    ├── books.py
    ├── members.py
    ├── loans.py
    └── reports.py
```

- The .venv folder stores the environment used by this project.
- Packages installed while it is active belong to that environment.
- Your source code remains outside .venv.
- Do not write your application code inside the .venv folder.

Common convention: name the folder .venv

Step 1 — Create the Virtual Environment in VS Code

Open the project folder, then open Terminal → New Terminal.

VS Code Terminal

```
python -m venv .venv
```

python	-m	venv	.venv
Python	run a module	virtual environment module	folder name

After running the command, VS Code will show a new .venv folder in the project.

Step 2 — Activate the Virtual Environment

Activation tells the terminal to use the project's environment.

Windows — Command Prompt

```
.venv\Scripts\activate
```

Windows — PowerShell

```
.venv\Scripts\Activate.ps1
```

Before activation

Terminal

```
C:\...\library_project>
```

After activation

Terminal

```
(.venv) C:\...\library_project>
```

The (.venv) prefix is a useful visual sign that the environment is active.

Step 3 — Install Packages Inside the Environment

Use pip after activating .venv.

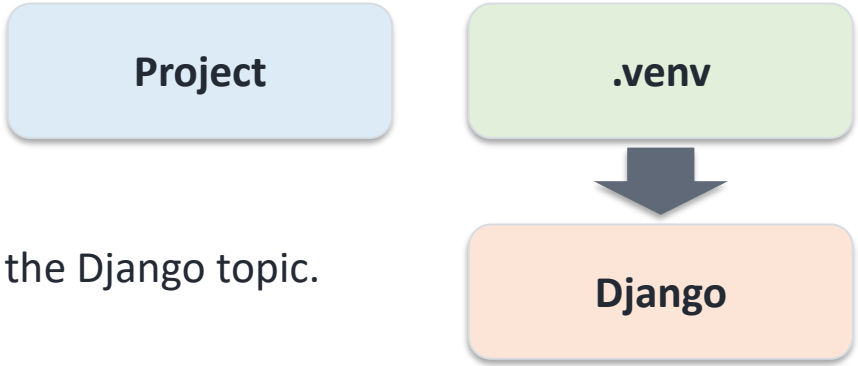
Install a package

```
python -m pip install django
```

See installed packages

```
python -m pip list
```

- `pip` is Python's package installer.
- Here Django is installed into the active virtual environment.
- Other projects do not automatically use this installation.
- Later, this is how students can prepare the environment for the Django topic.



Install dependencies only after activating the intended environment.

Step 4 — Select the Python Interpreter in VS Code

VS Code should run your code using the Python inside .venv.

- Open the Command Palette: Ctrl + Shift + P.
- Search for: Python: Select Interpreter.
- Choose the interpreter associated with your project's .venv.
- Now Run/Debug and the VS Code Python tools use that enviror

Ctrl + Shift + P

Python: Select Interpreter

Choose .venv

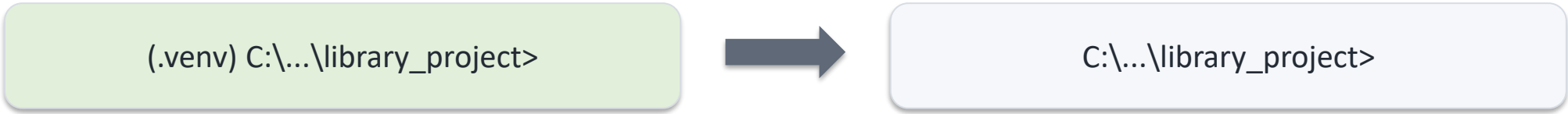
Important: activating the terminal and selecting the VS Code interpreter are related, but they are not exactly the same action.

Step 5 — Deactivate When Finished

Deactivate returns the terminal to its normal environment.

Terminal

```
deactivate
```



- Deactivation does not delete the virtual environment.
- The .venv folder remains in the project.
- You can activate it again whenever you continue working.

Virtual Environment vs Package vs pip

These terms perform different jobs.

Concept	Meaning	Library Project Example
Module	One Python file	loans.py
Package	Folder grouping Python modules	library/
Virtual environment	Isolated dependency environment	.venv/
pip	Installs Python packages/dependencies	pip install django

Do not confuse your own Python package (library/) with packages installed by pip.

Recommended Workflow for Every Python Project

Create → activate → install → code → deactivate

1. **Open/Create Project**
2. **Create .venv**
3. **Activate**
4. **Install Dependencies**
5. **Develop & Run**
6. **Deactivate**

python -m venv .venv

activate

python -m pip install ...

Good habit: create the virtual environment near the beginning of a new project.

Class Practical — Prepare the Library Project Environment

Students perform these steps in VS Code.

Expected structure

```
library_project/
├── .venv/
├── main.py
└── library/
    ├── __init__.py
    ├── books.py
    ├── members.py
    └── loans.py
```

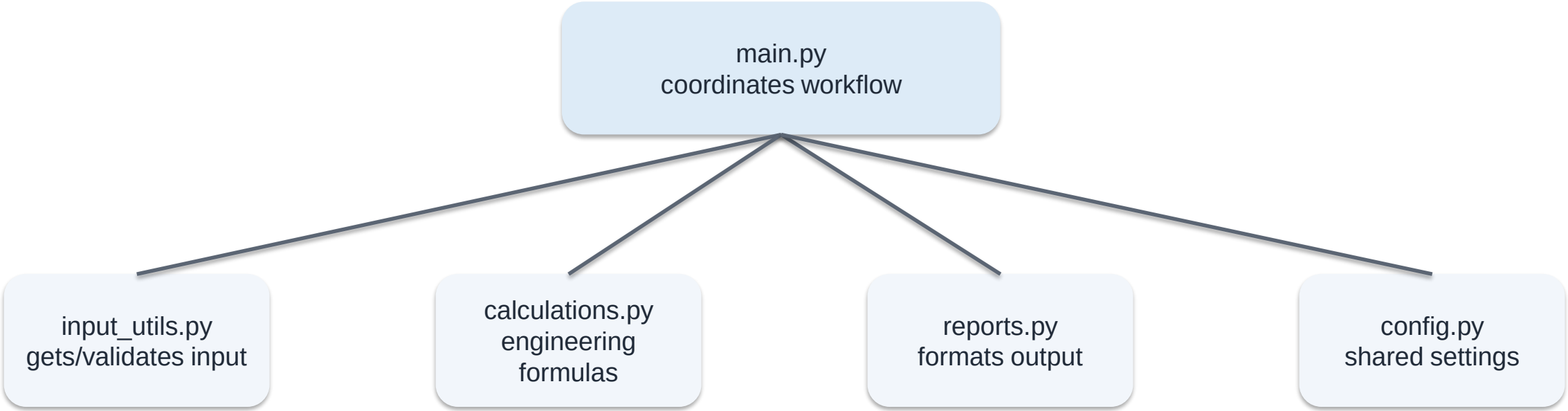
- Open library_project in VS Code.
- Create .venv using `python -m venv .venv`.
- Activate the environment.
- Confirm that `(.venv)` appears in the terminal.
- Run `python -m pip list`.
- Select the .venv Python interpreter in VS Code.
- Run main.py, then deactivate the environment.

Students should be able to explain WHY each step is performed—not only type the commands.

Organising a Multi-File Program

Organisation

Separate concerns instead of placing everything in main.py.



main.py should coordinate—not contain every detail.

Integrated Example: Library Loan & Fine Calculator

Practice

Combine functions, validation, modules and formatted output.

Python

```
def calculate_fine(days_late, rate=1000):
    return days_late * rate

def loan_status(days_late):
    return "Overdue" if days_late > 0 else "On time"

def main():
    member = input("Member name: ")
    book = input("Book title: ")
    days = int(input("Days late: "))
    fine = calculate_fine(days)
    status = loan_status(days)
    print(f"Member: {member}")
    print(f"Book: {book}")
    print(f"Status: {status}")
    print(f"Fine: UGX {fine}")

if __name__ == "__main__":
    main()
```

Ask students to identify inputs, functions and outputs.

Why does calculate_energy() receive power instead of recalculating it?

How would you validate negative values?

Which functions could be moved into calculations.py?

Guided Practical: Library Management Program

Practice

Design first; code second.

Problem: Develop a small library program that manages books and borrowing.
Required functions:
• register_member() — capture member details
• add_book() — add a title to the collection
• check_availability(books, title) — determine whether a book is available
• borrow_book(...) — process a borrowing transaction
• calculate_fine(days_late, rate=1000) — return an overdue fine
• display_library_report(...) — present results
Extension: place reusable functions in a separate module.

Design question:
Which functions calculate data and which functions handle presentation?

Do not give students every line of code. Let them decide how the functions cooperate.

Common Function Errors

Use these mistakes as live debugging demonstrations.

- Forgetting parentheses when calling: `greet` instead of `greet()`.
- Incorrect indentation inside the function.
- Supplying too many or too few arguments.
- Confusing parameter names with argument values.
- Forgetting to `return` a result.
- Expecting `print()` to behave like `return`.
- Trying to access a local variable outside its function.
- Accidentally shadowing a built-in name, e.g. `sum = 10`.

Debug me

```
def add(a, b):
    result = a + b
    # forgot return

answer = add(2, 3)
print(answer)   # None
```

Ask: Why is the output None?

Documenting Functions with Docstrings

Professional Practice

A docstring explains the purpose and expected use of a function.

Python

```
def calculate_power(voltage, current):
    """Calculate electrical power in watts.

    Args:
        voltage: Voltage in volts.
        current: Current in amperes.

    Returns:
        Power in watts.
    """
    return voltage * current
```

Place a triple-quoted string immediately inside the function.

State what the function does—not every implementation detail.

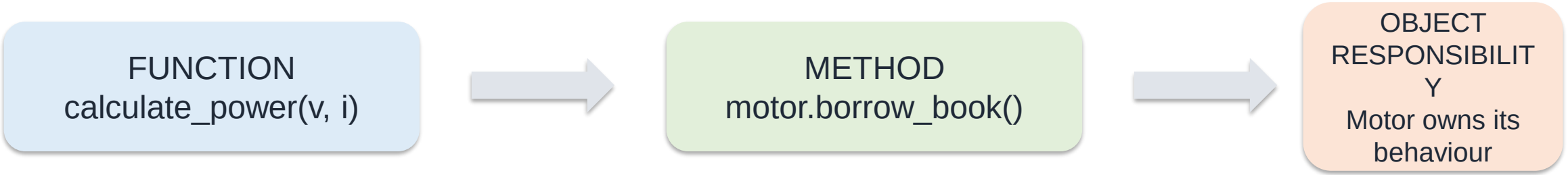
Document important parameters and return values.

Good documentation becomes increasingly important in modules and team projects.

Why Functions Matter for OOP

Bridge to OOP

Functions are a bridge to methods and class responsibilities.



Students who understand parameters, return values, scope and modularity will understand methods more easily.

OOP will combine related data and functions (methods) inside classes.

Good procedural decomposition therefore prepares students for good object-oriented design.

Knowledge Check

Review

Use these questions before the practical or at the end of the lecture.

- What is the difference between a parameter and an argument?
- What is the difference between `print()` and `return`?
- What happens to a local variable after its function finishes?
- When would you use a default parameter?
- Compare `import math` with `from math import sqrt`.
- What is the difference between a module and a package?
- Why should we avoid unnecessary global variables?
- What is the purpose of `if __name__ == '__main__':`?
- Why are virtual environments useful?
- How do functions prepare us for object-oriented programming?

Independent Practical Assignment

Practice

Library Management Toolkit

Develop a Python Library Management Toolkit that manages books, members and borrowing.
Your solution must contain at least five user-defined functions.
At least one function must accept multiple parameters and return a value.
Use at least one default parameter.
Use at least one appropriate standard-library module.
Separate reusable library functions into a user-defined module.
Use a main() function and the __name__ == '__main__' pattern.
Produce clearly formatted output and meaningful function names.
Extension: add validation and document each function using docstrings.

Required features
- Register a member
- Add/search books
- Check availability
- Borrow/return a book
- Calculate overdue fines

Assessment emphasis: correctness + decomposition + readability + reuse